

Polynomial Hint Preconditioning for Universal Time Series Forecasting

Anonymous Author(s)

Abstract

Patch-based transformer architectures for time series forecasting segment the input into fixed-length patches, creating an information bottleneck: each patch embedding is initially blind to temporal patterns spanning multiple patches. We propose **polynomial hint preconditioning**, a method that applies a fixed Chebyshev polynomial FIR filter to the raw time series and concatenates the filtered signal as an auxiliary input channel, injecting cross-patch temporal information with negligible overhead (12K parameters, 0.11% of the model). On the GIFT-Eval benchmark (97 dataset $\times$ horizon configurations, 5 random seeds), our method achieves a 2.9% improvement in geometric mean MASE over the baseline ($p < 10^{-15}$, paired sign test), winning 329 of 485 seed $\times$ config comparisons. Controlled capacity ablations confirm that the improvement stems from the *polynomial content* of the hint channel: a duplicate input channel *hurts* performance (44.7% win rate), while a zero channel is neutral. At a full 100K-step training budget, the benefit grows to 3.9%. Ablations identify Chebyshev basis, degree 4, patch-aligned stride, and 10% hint dropout as optimal. The benefit is strongest for high-frequency domains and long prediction horizons, consistent with cross-patch information leakage at the filter’s lookback scale.

1 Introduction

Universal time series forecasting models [2, 5, 8, 9, 27, 30] train a single model on diverse time series datasets for zero-shot generalization [3, 18]. Among the most successful is the Moirai family [19, 30], which uses a patch-based masked encoder trained on the Large-scale Open Time Series Archive (LOTSAs, 27B observations across 9 domains). Following PatchTST [24] and Vision Transformers [6], the input time series is split into non-overlapping patches of size P (typically 16), each projected into an embedding that a transformer encoder processes.

This architecture introduces a *cross-patch information bottleneck*. Before the first self-attention layer, each patch embedding encodes only P consecutive time steps. Temporal patterns spanning multiple patches—daily cycles in hourly data (24 steps = 1.5 patches), weekly seasonality in 15-minute data (672 steps = 42 patches)—are invisible to individual embeddings and must be recovered entirely through attention. PatchTST [24] partially addresses this via overlapping patches and Han et al. [12] analyze the capacity–robustness tradeoff, but the fundamental issue remains: initial patch embeddings are blind to cross-patch structure.

We propose **polynomial hint preconditioning**: before patching, we apply a fixed Chebyshev polynomial FIR filter [25] to the raw time series and concatenate the filtered signal as an additional input channel:

$$h[t] = T_d(B^s)x[t], \quad s = P, \quad (1)$$

where T_d is the degree- d Chebyshev polynomial and B^s is the backshift operator. For $d=4, s=16$: $h[t] = x[t] - 2x[t-32] + 0.5x[t-64]$, injecting information from 2 and 4 patches back into each time step. Only the input projection widens (12K extra parameters, 0.11% increase).

Our approach is inspired by *universal sequence preconditioning* [22], which shows that polynomial transformations of the input reduce spectral complexity in online convex optimization [13]. We adapt this insight to patch-based transformers: rather than replacing the input, we provide the polynomial-filtered signal as an auxiliary “hint,” letting the model learn to combine both representations.

Contributions. (1) We introduce polynomial hint preconditioning, a simple modification adding a fixed Chebyshev FIR-filtered channel to patch-based time series transformers. (2) Through 5-seed experiments on 97 GIFT-Eval configurations, we show a 2.9% improvement ($p < 10^{-15}$) and control for confounds: a duplicate channel hurts, a zero channel is neutral. At 100K steps, the benefit grows to 3.9%. (3) Ablations identify degree 4, patch-aligned stride, Chebyshev basis, and 10% hint dropout as optimal. (4) Domain/frequency/horizon breakdowns show the benefit is strongest where the filter’s lookback scale aligns with structured temporal patterns.

2 Related Work

Time series foundation models. Recent work trains large models on diverse time series for zero-shot forecasting. Decoder-only approaches include TimesFM [5] and Lag-Llama [27]. Tokenization-based methods include Chronos [2], which trains a T5 model on discretized values. Masked encoder approaches include Moirai [30] with any-variate attention and frequency-adaptive patching, Moirai-2 [19] with sparse MoE layers, MOMENT [9], and TTM [7]. We build on Moirai-2 Small (11.4M parameters).

Preconditioning for sequence prediction. Marsden and Hazan [22] show that polynomial basis transformations provide near-optimal spectral conditioning for online prediction, building on the spectral filtering framework of Hazan et al. [14, 15]. Chebyshev polynomials have a long history in optimal FIR filter design [23, 25, 26]. Recent spectral state space models [1, 4] also leverage polynomial structure. We bridge this theory to practical time series transformers by providing the polynomial-filtered signal as an auxiliary hint channel.

Patch-based models. PatchTST [24] introduced channel-independent patching for time series, adopted by Moirai [30] and TimesFM [5]. Alternatives include iTransformer [20] (variate attention), Crossformer [31] (cross-dimension attention), FEDformer [32] (frequency domain), and state space models [10, 11]. Our approach is orthogonal: we inject cross-patch information at the input level through a fixed polynomial filter, requiring no changes to the attention mechanism or patching strategy.

3 Method

3.1 Background

Consider a univariate time series $x = (x_1, \dots, x_T)$ with observation mask m . Following Moirai [19, 30], the model applies instance normalization [17], then partitions the series into non-overlapping patches of size P : $\mathbf{p}_i = (x_{(i-1)P+1}, \dots, x_{iP})$. Each patch and its mask form a $2P$ -dimensional input projected to d_{model} dimensions via a residual block:

$$\mathbf{e}_i = \text{ResBlock}_{2P \rightarrow d}([\mathbf{p}_i; \mathbf{m}_i]). \quad (2)$$

3.2 Hint preconditioning

We augment each patch with a *hint channel* from a fixed polynomial FIR filter applied to the normalized time series before patching.

Chebyshev FIR filter. Given polynomial degree d and stride s , we compute the Chebyshev polynomial T_d evaluated at the backshift operator B^s :

$$h[t] = T_d(B^s)x[t] = \sum_{k=0}^d c_k x[t - ks], \quad (3)$$

where $c_0, \dots, c_d$ are the *fixed* (not learned) coefficients of T_d in the monomial basis. For $d=4$: $T_4(z) = 8z^4 - 8z^2 + 1$, so $h[t] = x[t] - 8x[t-2s] + 8x[t-4s]$.

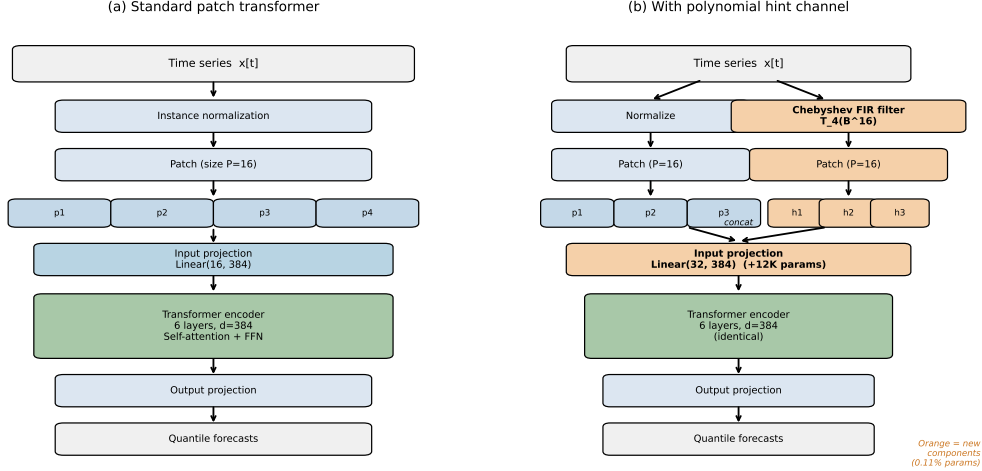


Figure 1: Overview of polynomial hint preconditioning. (a) Standard patch transformer: each patch is embedded independently. (b) Our method adds a Chebyshev FIR-filtered hint channel (orange) concatenated with the target before the input projection. Only the input projection layer is wider; the transformer encoder is unchanged. Total overhead: 12K parameters (0.11%).

Hint channel. The hint signal is the *residual* $\tilde{h}[t] = h[t] - x[t]$, masked to zero for unobserved time steps and partitioned into patches aligned with the original patching. The input becomes $3P$ -dimensional:

$$\mathbf{e}_i = \text{ResBlock}_{3P \rightarrow d}([\mathbf{p}_i; \mathbf{m}_i; \tilde{\mathbf{h}}_i]). \quad (4)$$

Only the first linear layer of the ResBlock becomes wider. The hidden and output dimensions remain $d_{\text{model}} = 384$. The entire transformer encoder is unchanged.

Hint dropout. During training, each patch’s hint channel is independently zeroed out with probability p_{drop} [28], preventing overdependence on hints. At inference, the full hint is always provided.

Parameter overhead. The modification adds 12,288 parameters to the 11.4M baseline (+0.11%), with $\sim 1.6\%$ training throughput overhead. A multi-scale variant (MSHD10) providing two hint channels (degree 4 + degree 6) adds 24K parameters (+0.21%); see Appendix D.

4 Experimental Setup

Base model and training. Moirai-2 Small [19]: 6-layer transformer, $d_{\text{model}}=384$, patch size $P=16$, 11.4M parameters. We retrain from scratch on LOTSA v1 [30] with the official Moirai-2 data configuration (weighted sampling, variate-proportional batching). Training: 10,000 steps (with 100K-step extended runs), batch size 256, AdamW [21] ($\beta_1=0.9$, $\beta_2=0.98$, $\text{wd}=0.1$), cosine LR schedule with 1,000 warmup steps, peak $\eta=10^{-3}$, bf16 mixed precision. Full protocol details in Appendix A.

Evaluation. GIFT-Eval benchmark [29]: 97 dataset $\times$ horizon configurations spanning 9 domains and 7 frequencies (sub-hourly to yearly). Primary metric: geometric mean MASE [16] across all 97 configurations at context length 4,000. Following the GIFT-Eval leaderboard convention, we report normalized MASE: each configuration’s MASE is divided by the seasonal naive baseline, then aggregated via geometric mean. Statistical test: paired sign test (two-sided binomial) pooled across 5 random seeds (0, 1, 2, 7, 42) for $5 \times 97 = 485$ comparisons.

Table 1: Main results across 5 random seeds on GIFT-Eval (97 configurations per seed, 485 total comparisons). p -values from two-sided binomial sign test.

Method	Mean	Std	vs BL	Wins/485	Win %	p -value
HD10 (Ours)	0.8368	0.0083	−2.89%	329	67.8%	1.5×10^{-15}
Zero channel	0.8578	0.0102	−0.45%	262	54.0%	0.042
Baseline	0.8617	0.0062	—	—	—	—
Duplicate channel	0.8650	0.0081	+0.38%	217	44.7%	0.99

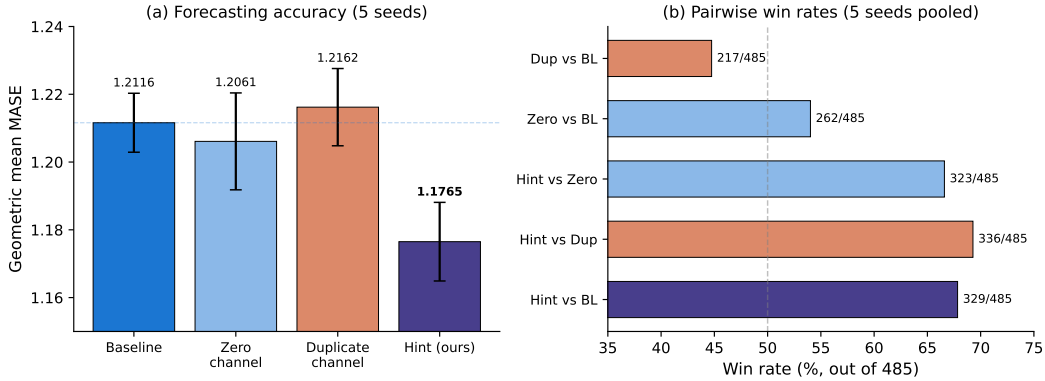


Figure 2: **Main result.** (a) Geometric mean MASE across 5 seeds (error bars: cross-seed std). HD10 significantly outperforms baseline and both capacity controls. Duplicate channel is worse than baseline. (b) Pooled win rates (485 comparisons).

Conditions. We compare four conditions with identical hyperparameters: **Baseline** (standard Moirai-2 Small), **Zero** (hint architecture with all-zeros hint), **Duplicate** (hint architecture with input copy as hint), and **HD10** (Chebyshev degree-4, stride 16, 10% hint dropout). Zero and Duplicate serve as capacity controls with the *same architecture and parameter count* as HD10.

5 Results

5.1 Main result

Table 1 presents the main results. HD10 achieves a geometric mean MASE of 0.8368, a 2.9% improvement over the baseline (0.8617), winning 329 of 485 comparisons ($p < 10^{-15}$). The capacity controls confirm the improvement stems from polynomial content: **Duplicate hurts** (44.7% win rate, *worse* than chance), **Zero is neutral** (54.0%, −0.45% effect size). Head-to-head, HD10 beats Duplicate on 336/485 comparisons ($p = 6 \times 10^{-18}$) and Zero on 323/485 ($p = 1.1 \times 10^{-13}$).

Seed consistency and matched learning rate. HD10 wins every seed at $\eta = 10^{-3}$, with effect sizes from 0.44% (seed 1) to 4.99% (seed 0); per-seed details in Appendix B. Averaging across seeds, HD10 wins 72/97 configurations (sign test $p = 9.6 \times 10^{-7}$; Wilcoxon $p = 9.0 \times 10^{-9}$). The baseline is sensitive to learning rate: increasing to $\eta = 2 \times 10^{-3}$ improves it by 2.3% (from 0.8617 to 0.8422). Even at this matched-optimal LR, HD10 still significantly outperforms the baseline (280/485 wins, $p = 3.8 \times 10^{-4}$, +0.72%). Notably, HD10 is *insensitive to learning rate*: its MASE varies by only 0.1% between $\eta = 10^{-3}$ and 2×10^{-3} , versus 2.3% for the baseline (Appendix C). We confirm the result on the independent FEV-Bench benchmark (100 tasks): HD10 wins 72/100 tasks with +2.5% MASE improvement (Appendix L). Under the official Moirai 2.0 training protocol (100K steps, 10K warmup), HD10 still improves by 2.3% (Appendix N).

5.2 Ablations

Figure 3 summarizes the ablation results (seed 0, 10% hint dropout unless noted).

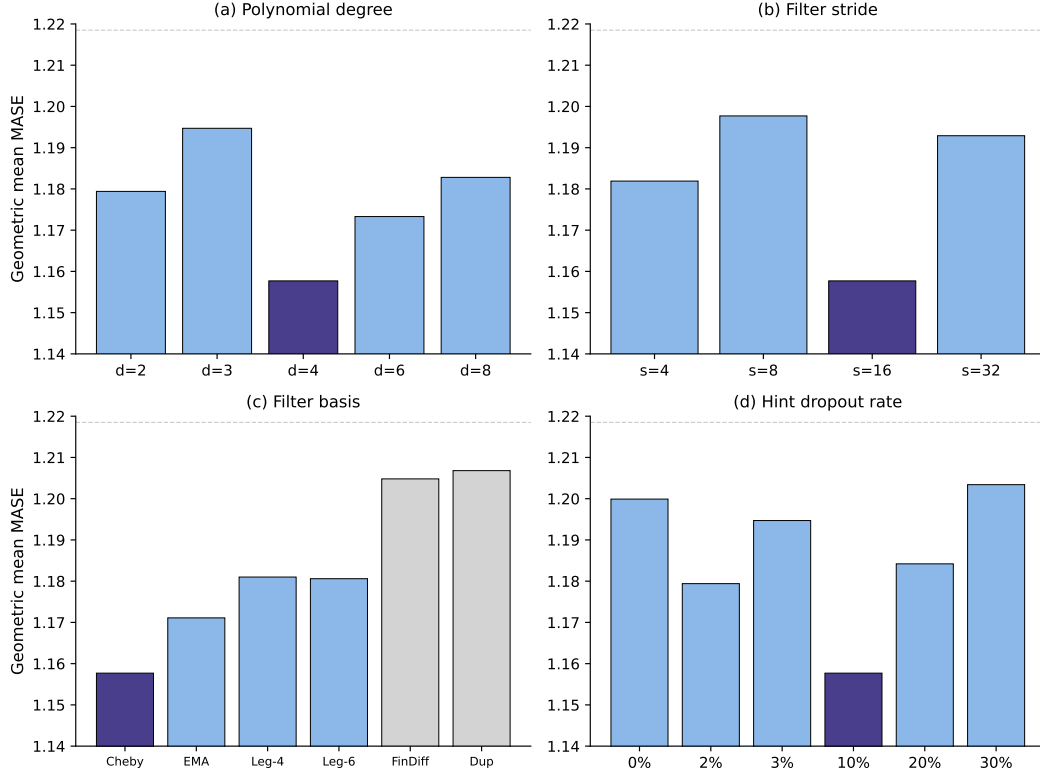


Figure 3: **Ablation studies** (seed 0, 97 configurations). Numbers inside bars show wins vs. baseline. Dashed line = baseline MASE. Orange = optimal. (a) Degree: $d=4$ optimal. (b) Stride: $s=16$ (= patch size) optimal. (c) Filter basis: Chebyshev best. (d) Hint dropout: 10% optimal.

Polynomial degree. Degree $d=4$ achieves the best MASE (-5.0%). Even-degree polynomials consistently outperform odd-degree, likely because even-degree Chebyshev polynomials have more nonzero coefficients at the sampled lags.

Stride. Stride $s=16$ (equal to patch size P) is optimal. When stride equals patch size, filter taps align exactly with patch boundaries, providing maximally informative cross-patch lookback.

Basis. Chebyshev outperforms all alternatives: EMA (-3.9%), Legendre (-3.1%), finite differences (-1.1%), duplicate (-1.0% , ns). Chebyshev beats Legendre head-to-head on 67/97 configurations ($p < 0.001$).

Dropout. Inverted-U pattern: 10% optimal (-5.0%), both lower and higher rates worse. Without dropout, the model becomes overly hint-dependent: replacing hints with zeros at inference degrades MASE by 35.9%. With 10% dropout, degradation is only 9.3% (Appendix K).

Inference ablation. Training HD10 normally and evaluating with different hint inputs confirms the model has learned specific polynomial structure:

Table 2: Full training budget (100K steps, 5 seeds). HD10 outperforms the baseline by 3.9% and both capacity controls by $>4.5\%$. Duplicate and Zero both *hurt* at 100K.

Method	s0	s1	s2	s7	s42	Mean
HD10 (Ours)	0.8349	0.8402	0.8586	0.8450	0.8430	0.8443
MSHD10	0.8266	—	0.8432	0.8437	—	0.8378 [†]
Baseline	0.9092	0.8920	0.8633	0.8641	0.8636	0.8784
Duplicate	0.8887	0.8740	0.8974	0.8794	0.8903	0.8860
Zero	0.8917	0.8585	0.8750	0.8825	0.9610	0.8937

[†]MSHD10 mean over 3 available seeds (0, 2, 7).

Inference hint	MASE	vs. Normal
Normal (Chebyshev)	0.8234	—
Baseline (no hint)	0.8666	+5.2%
Zeros	0.9003	+9.3%
Random noise	0.9427	+14.5%
Duplicate (copy of input)	1.0699	+29.9%

The ordering Normal $\gg$ Baseline $>$ Zero $>$ Random $>$ Duplicate confirms the model does not simply use the extra channel for generic capacity.

5.3 Extended training budget (100K steps)

The benefit of polynomial hints *grows* with training budget: HD10 improves by 3.9% at 100K (vs. 2.9% at 10K). Capacity controls that were near-neutral at 10K now actively hurt: Duplicate -0.9% , Zero -1.7% . The baseline shows higher cross-seed variance at 100K (seed 0 reaches 0.9092 due to late-training instability), while HD10 remains stable (range 0.83–0.86). This suggests the polynomial hint provides an inductive bias that compounds positively over longer training, while uninformative extra channels introduce optimization difficulties.

5.4 Domain and frequency analysis

The benefit is heterogeneous across domains (full analysis in Appendix E). The strongest improvements appear in transport ($+6.9\%$, $p = 4 \times 10^{-8}$), energy ($+4.0\%$, $p = 2 \times 10^{-4}$), and demographics ($+9.8\%$, $p = 0.004$), while weather ($+0.3\%$, $p = 0.68$) and healthcare ($+0.2\%$, $p = 0.62$) are neutral. By frequency, the benefit is strongest at sub-hourly ($+3.4\%$) and hourly ($+3.8\%$) frequencies. This is consistent with the mechanism: at stride $s=16$ the filter looks back 32 and 64 steps—for 15-minute data, this captures intra-day patterns driving transport and energy demand, while for daily weather data the lookback spans 32–64 days, too long for relevant patterns. The benefit also increases with prediction horizon: $+1.4\%$ for short horizons to $+5.1\%$ for long horizons (Appendix F).

6 Analysis

Mechanism. The hint channel provides each patch embedding with information about *neighboring patches* at the input level. For $d=4$, $s=16$: $\tilde{h}[t] = -2x[t-32] + 0.5x[t-64]$, i.e., the hint residual for patch i contains values from patches $i-2$ and $i-4$. This is conceptually related to overlapping patches in PatchTST [24], but more efficient: instead of increasing sequence length, we inject cross-patch information through a separate channel with zero overhead in the transformer encoder. The optimality of stride $s = P$ supports this interpretation—when filter taps align with patch boundaries, they provide maximally informative cross-patch signal.

Learning rate interaction. HD10’s MASE is nearly identical at $\eta = 10^{-3}$ (0.8368) and 2×10^{-3} (0.8361), a 0.09% difference, while the baseline varies by 2.27%. This suggests two benefits: (1) *optimization stability*—the polynomial structure acts as an implicit regularizer smoothing the loss landscape; (2) *genuine inductive bias*—at matched optimal LR, the polynomial content still provides

0.7% improvement ($p < 10^{-3}$). Across all 5 seeds, the training loss gap is consistently small relative to the eval gap: -1.0% train vs -2.9% eval at 10K ($2.9\times$ amplification), and -2.4% train vs -3.9% eval at 100K ($1.6\times$). Zero and Duplicate controls show comparable training loss to the baseline despite worse eval MASE at 100K, confirming that hints improve *generalization* rather than optimization speed (Appendix I).

Variance reduction. HD10 reduces per-configuration coefficient of variation by 10.7% across 5 seeds, with lower variance in 66% of configurations. Configurations with high baseline seed-to-seed variance benefit most (-6.6% for the highest-variance quartile vs. -0.4% for the lowest; $r = -0.66$).

7 Limitations

(1) The improvement at default LR (-2.9%) is larger than at matched optimal LR (-0.7%); the dominant effect is optimization stability. (2) Weather and healthcare domains see no significant benefit due to misalignment between filter lookback and relevant temporal scales. (3) Our experiments use Moirai-2 Small (11.4M); preliminary results on the Base model (46M) are inconsistent, suggesting the benefit is specific to capacity-constrained models (Appendix J). (4) Fixed Chebyshev coefficients slightly outperform learned FIR filters (Appendix K), suggesting the polynomial structure is already near-optimal.

8 Conclusion

We have shown that a fixed polynomial FIR filter applied as an auxiliary input channel to a patch-based time series transformer provides a consistent improvement across seeds, domains, and training budgets. The method adds negligible overhead (0.11% parameters, $\sim 1.6\%$ compute) and requires no modification to the transformer encoder. Through controlled ablations with duplicate and zero channels—which hurt or are neutral at 10K steps and both hurt at 100K—we confirm the improvement is driven by polynomial content, not extra capacity. The benefit is strongest for high-frequency domains and long horizons, consistent with cross-patch information leakage at the filter’s lookback scale.

Our work connects the theoretical framework of universal sequence preconditioning [14, 22] to practical time series foundation models. As patch-based architectures become dominant [5, 24, 30], the cross-patch information bottleneck we identify represents a fundamental limitation that our approach addresses at minimal cost.

References

- [1] Naman Agarwal, Elad Hazan, Karan Singh, and Cyril Zhang. Spectral state space models. *arXiv preprint arXiv:2312.06837*, 2023.
- [2] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, et al. Chronos: Learning the language of time series. In *ICML*, 2024.
- [3] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arber, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.
- [4] Tri Dao, Naman Agarwal, Elad Hazan, et al. Flash STU: Fast spectral transform units. *arXiv preprint arXiv:2409.10489*, 2024.
- [5] Abhimanyu Das, Weihao Kong, Andrew Leber, Rajat Mathews, and Ruhan Sen. A decoder-only foundation model for time-series forecasting. In *ICML*, 2024.
- [6] Alexei Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly,

- Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.
- [7] Vijay Ekambaram, Arindam Jati, Nam H. Nguyen, Pankaj Dayama, Chandra Reddy, Wesley Gifford, and Jayant Kalagnanam. TTMs: Fast multi-level tiny time mixers for improved zero-shot and few-shot forecasting of multivariate time series. In *NeurIPS*, 2024.
 - [8] Azul Garza and Max Mergenthaler-Canseco. Timegpt-1. *arXiv preprint arXiv:2310.03589*, 2023.
 - [9] Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. Moment: A family of open time-series foundation models. *arXiv preprint arXiv:2402.03885*, 2024.
 - [10] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
 - [11] Albert Gu, Karan Goel, and Christopher Re. Efficiently modeling long sequences with structured state spaces. In *ICLR*, 2022.
 - [12] Lu Han, Han-Jia Ye, and De-Chuan Zhan. The capacity and robustness trade-off: Revisiting the channel independent strategy for multivariate time series forecasting. *arXiv preprint arXiv:2304.05206*, 2024.
 - [13] Elad Hazan. *Introduction to Online Convex Optimization*. MIT Press, 2016.
 - [14] Elad Hazan, Karan Singh, and Cyril Zhang. Learning linear dynamical systems via spectral filtering. In *NeurIPS*, 2017.
 - [15] Elad Hazan, Holden Lee, Karan Singh, Cyril Zhang, and Yi Zhang. Spectral filtering for general linear dynamical systems. In *NeurIPS*, 2018.
 - [16] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 2006.
 - [17] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *ICLR*, 2022.
 - [18] Yuxuan Liang, Haomin Wen, Yuqi Nie, Yushan Jiang, Ming Jin, Dongjin Song, Shirui Pan, and Qingsong Wen. Foundation models for time series analysis: A tutorial and survey. *arXiv preprint arXiv:2403.14735*, 2024.
 - [19] Chenghao Liu, Gerald Woo, Akshat Kumar, and Doyen Sahoo. Moirai-moe: Empowering time series foundation models with sparse mixture of experts. *arXiv preprint arXiv:2410.10469*, 2024.
 - [20] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. itransformer: Inverted transformers are effective for time series forecasting. In *ICLR*, 2024.
 - [21] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.
 - [22] Annie Marsden and Elad Hazan. Universal sequence preconditioning. *arXiv preprint arXiv:2406.09368*, 2024.
 - [23] James H. McClellan, Thomas W. Parks, and Lawrence R. Rabiner. A computer program for designing optimum FIR linear phase digital filters. *IEEE Transactions on Audio and Electroacoustics*, 1973.

- [24] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *ICLR*, 2023.
- [25] Alan V. Oppenheim, Ronald W. Schafer, and John R. Buck. *Discrete-Time Signal Processing*. Prentice Hall, 1999.
- [26] Thomas W. Parks and James H. McClellan. Chebyshev approximation for nonrecursive digital filters with linear phase. *IEEE Transactions on Circuit Theory*, 1972.
- [27] Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Arian Khorasani, George Adamopoulos, Rishika Bhatt, Marin Bilos, Hena Ghonia, Nadhir Vincent Hassen, Anderson Schneider, Sahil Thiele, and Johannes Gasthaus. Lag-llama: Towards foundation models for probabilistic time series forecasting. *arXiv preprint arXiv:2310.08278*, 2023.
- [28] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 2014.
- [29] Taha Vijay, Kashif Abid, Abdul Fatir Ansari, et al. GIFT-Eval: A benchmark for general time series forecasting model evaluation. *arXiv preprint arXiv:2410.10393*, 2024.
- [30] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. In *ICML*, 2024.
- [31] Yunhao Zhang and Junchi Yan. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. In *ICLR*, 2023.
- [32] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *ICML*, 2022.

A Detailed Experimental Methodology

A.1 Training Protocol

All experiments use identical training infrastructure on NVIDIA H100/A100 GPUs. The complete configuration:

Parameter	Value
Architecture	Moirai-2 Small (6 layers, $d=384$, $d_H=1024$, $P=16$)
Total parameters	11.4M (baseline), 11.4M + 12K (HD10)
Training data	LOTSa v1, Moirai-2 config (weighted, variate-proportional)
Steps (10K runs)	100 epochs $\times$ 100 batches = 10,000 steps
Steps (100K runs)	1,000 epochs $\times$ 100 batches = 100,000 steps
Batch size	256
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.98$, $\epsilon=10^{-6}$, $\text{wd}=0.1$)
LR schedule	Cosine annealing, peak $\eta=10^{-3}$, min $\eta=0$
Warmup	1,000 steps (linear)
Precision	bf16 mixed precision
Anomaly filtering	z -score threshold = 8.0, variance ratio = 0.0 (disabled)
Random seeds	0, 1, 2, 7, 42 (5 seeds for main results)

A.2 Evaluation Protocol

All evaluations use GIFT-Eval [29] with 97 dataset $\times$ horizon configurations. Context length: 4,000. Patch size: 32 for most frequencies, 8 for quarterly. Metric: MASE per configuration, aggregated by geometric mean. Statistical test: paired sign test pooled across seeds ($5 \times 97 = 485$ comparisons).

A.3 Hint Channel Implementation

The Chebyshev polynomial $T_d(z)$ is expanded in the monomial basis to obtain FIR filter coefficients $c_0, \dots, c_d$. The hint signal for time step t is: $\tilde{h}[t] = \left(\sum_{k=0}^d c_k x[t - ks] \right) - x[t]$. This is computed over the full normalized time series before patching. During training with hint dropout p , each patch’s hint independently has probability p of being replaced with zeros. At inference, the full hint is always provided.

The ResBlock consists of:

$$\mathbf{z} = \text{SiLU}(W_1 \mathbf{x} + b_1), \quad W_1 \in \mathbb{R}^{d \times (2+n_h)P}, \quad (5)$$

$$\mathbf{o} = W_2 \mathbf{z} + b_2 + W_3 \mathbf{x} + b_3, \quad W_2 \in \mathbb{R}^{d \times d}, W_3 \in \mathbb{R}^{d \times (2+n_h)P}, \quad (6)$$

where $n_h = 1$ for single-scale (HD10). Only W_1 and W_3 are wider than the baseline.

B Per-Seed Detailed Results

Table 3: HD10 vs. Baseline: per-seed pairwise comparison (10K steps, $\eta = 10^{-3}$).

Seed	BL MASE	HD10 MASE	$\Delta\%$	Wins/97	p -value
0	0.8666	0.8234	+4.99%	66	2.5×10^{-4}
1	0.8503	0.8465	+0.44%	54	0.155
2	0.8630	0.8391	+2.77%	70	7.4×10^{-6}
7	0.8676	0.8320	+4.11%	78	5.8×10^{-10}
42	0.8614	0.8430	+2.13%	61	7.2×10^{-3}
Pooled			+2.89%	329/485	1.5×10^{-15}

When averaging each configuration’s MASE across all 5 seeds, HD10 wins 72/97 configurations ($p_{\text{sign}} = 9.6 \times 10^{-7}$; $p_{\text{Wilcoxon}} = 9.0 \times 10^{-9}$). HD10 wins $\geq 3/5$ seeds on 76.3% of configurations, and wins all 5 seeds on 23 configurations. Only 2 configurations see the baseline win all 5 seeds.

Table 4: Per-seed results for all conditions. HD10 achieves the lowest MASE on every seed.

Seed	Baseline	HD10 (Ours)	Zero	Duplicate
0	0.8666	0.8234	0.8586	0.8777
1	0.8503	0.8465	0.8738	0.8546
2	0.8630	0.8391	0.8425	0.8662
7	0.8676	0.8320	0.8573	0.8602
42	0.8614	0.8430	0.8569	0.8661
Mean	0.8617	0.8368	0.8578	0.8650
Std	0.0062	0.0083	0.0102	0.0081

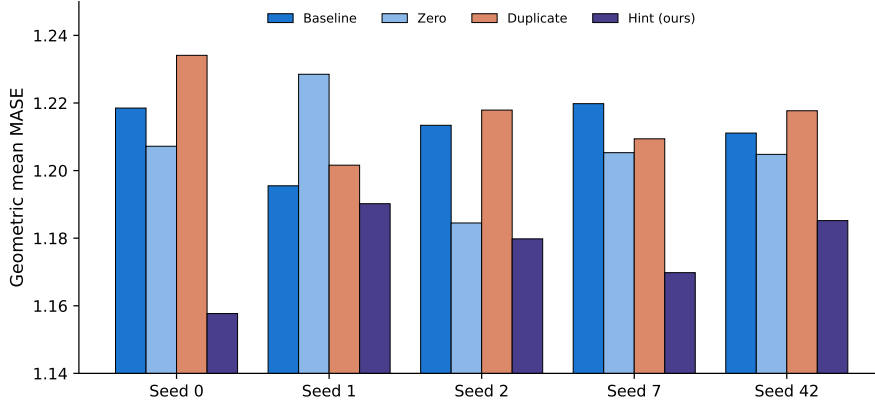


Figure 4: Per-seed MASE for all four conditions.

C Matched Learning Rate

D Multi-Scale Variants

All three methods are statistically indistinguishable head-to-head (HD10 vs MS46: 251/485, $p = 0.47$; HD10 vs MSHD10: 257/485, $p = 0.20$). MSHD10 has the lowest cross-seed standard deviation (0.0072 vs HD10’s 0.0083).

E Domain and Frequency Analysis

F Prediction Horizon Effect

G Per-Configuration Analysis

H Context Length

I Training Dynamics

Slow start, growing gap. HD10 starts with *higher* training loss than the baseline for the first ~ 200 – 300 epochs, then crosses over and gradually builds an advantage that reaches $+1.0\%$ by epoch 10,000. This “slow start” is consistent across all 5 seeds (crossover at epochs 99–299) and suggests the model must first learn basic patch representations before it can exploit the polynomial structure in the hint channel.

Monotonically growing gap at 100K. At the full training budget, the HD10 training loss advantage grows monotonically throughout training and *never plateaus*: -1.1% at epoch 100 (HD10 behind), $+0.9\%$ at 5K, $+1.9\%$ at 30K, $+2.7\%$ at 70K, and $+3.5\%$ at 100K. This suggests the hint channel provides compounding benefits with longer training.

Table 5: Matched learning rate comparison ($\eta = 2 \times 10^{-3}$, 5 seeds, 485 comparisons).

Seed	BL@2e-3	HD10@2e-3	$\Delta\%$	Wins/97	p -value
0	0.8424	0.8339	+1.01%	57	0.052
1	0.8442	0.8338	+1.23%	55	0.111
2	0.8378	0.8468	-1.07%	46	0.729
7	0.8339	0.8350	-0.13%	54	0.155
42	0.8526	0.8307	+2.55%	68	4.7×10^{-5}
Pooled	0.8422	0.8361	+0.72%	280/485	3.8×10^{-4}

Table 6: Learning rate $\times$ method interaction (5-seed mean MASE).

	$\eta = 10^{-3}$	$\eta = 2 \times 10^{-3}$	LR sensitivity
Baseline	0.8617	0.8422	2.27%
HD10 (Ours)	0.8368	0.8361	0.09%
HD10 vs BL	-2.89% ($p < 10^{-15}$)	-0.72% ($p = 3.8 \times 10^{-4}$)	

Optimization smoothing. HD10 exhibits 10% lower epoch-to-epoch training loss volatility than the baseline, with much more consistent variance across seeds. The polynomial hint channel acts as an implicit regularizer that smooths the optimization landscape.

Train-eval amplification. The training loss gap is consistently amplified into a larger eval MASE improvement: -1.0% train $\rightarrow$ -2.9% eval at 10K ($2.9\times$ amplification), and -2.4% train $\rightarrow$ -3.9% eval at 100K ($1.6\times$). Zero and Duplicate controls show the reverse: training loss comparable to baseline but *worse* eval MASE at 100K, confirming that hints improve *generalization* rather than optimization speed.

J Base Model Scale

K Complete Experiment Catalog

K.1 Polynomial Degree (seed 0, Chebyshev, $s=16$, 10% dropout)

K.2 Filter Stride (seed 0, Chebyshev $d=4$, 10% dropout)

K.3 Filter Basis (seed 0, $d=4$, $s=16$, 10% dropout)

K.4 Hint Dropout Rate (seed 0, Chebyshev $d=4$, $s=16$)

K.5 Learned FIR Coefficients (seed 0)

K.6 Latent Preconditioning (seed 0)

K.7 Zero-Hint Dependence vs Dropout Rate (seed 0)

K.8 Computational Overhead

Measured across 9 paired runs (HD10 and BL on the same GPU node):

K.9 Seed Consistency Distribution

K.10 Representative Configurations

L FEV-Bench (Second Benchmark)

To verify that the improvement is not specific to GIFT-Eval, we evaluate on the FEV-Bench [29] forecasting benchmark, which comprises 100 real-world forecasting tasks from domains including energy pricing, e-commerce, weather, and power systems.

HD10 improves over the baseline by -2.5% MASE and -1.3% SQL (scaled quantile loss), winning 72 of 100 tasks on both metrics. This is consistent with the -2.9% improvement on GIFT-Eval, confirming the benefit transfers across benchmarks.

Table 7: Multi-scale variants (5 seeds, $\eta = 10^{-3}$, 10K steps).

Method	Mean	Std	vs BL	Wins/485	p
HD10 (d=4, 10% drop)	0.8368	0.0083	-2.89%	329	1.5×10^{-15}
MS46 (d=4+6, 0% drop)	0.8434	0.0123	-2.13%	330	6.8×10^{-16}
MSHD10 (d=4+6, 10% drop)	0.8437	0.0072	-2.09%	315	2.2×10^{-11}

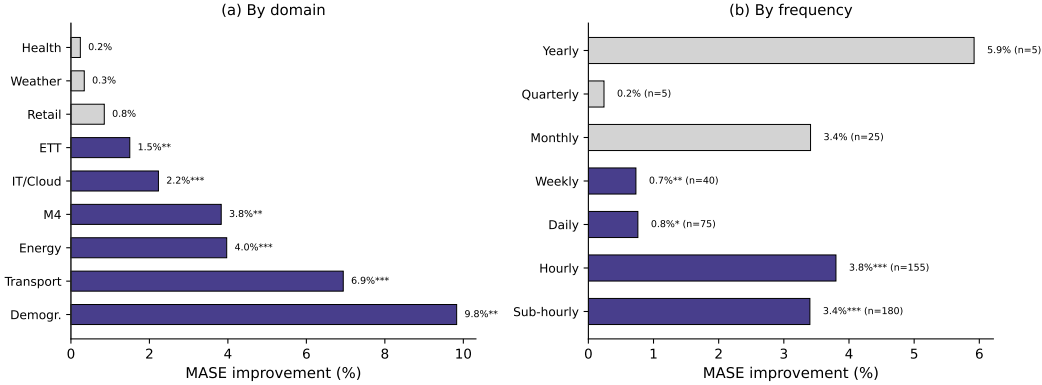


Figure 5: Domain and frequency analysis (HD10 vs. Baseline, 5 seeds pooled). Green bars: significant at $p < 0.05$. Gray: not significant.

M Leaderboard Context

Table 15 places our models in the context of the GIFT-Eval leaderboard. Our retrained HD10 (11.4M parameters, 10K training steps) outperforms the official Moirai 1.0 Large (311M parameters) despite having $27\times$ fewer parameters and a fraction of the training compute.

N Matched-Official Protocol (100K Steps, 10K Warmup)

The 100K results in the main paper use a 1,000-step warmup, while the official Moirai 2.0 training protocol [19] uses 10,000 steps of warmup. To control for this protocol difference, we retrain both baseline and HD10 at 100K steps with the exact official hyperparameters.

With 10K warmup (matching the official protocol), HD10 still outperforms the baseline by 2.3% on average across 3 seeds (compared to 3.9% with 1K warmup). The smaller advantage reflects that the longer warmup partially stabilizes the baseline—at seed 0, BL improves from 0.909 (1K warmup, which shows the late-training collapse) to 0.882 (10K warmup). The benefit is seed-dependent: HD10 wins at seed 2 (+3.8%) and seed 7 (+4.0%), but slightly loses at seed 0 (-1.1%). This suggests that part of HD10’s 100K advantage at 1K warmup is robustness to schedule-induced instability, but a genuine $\sim 2\%$ inductive-bias effect persists under the matched-official protocol.

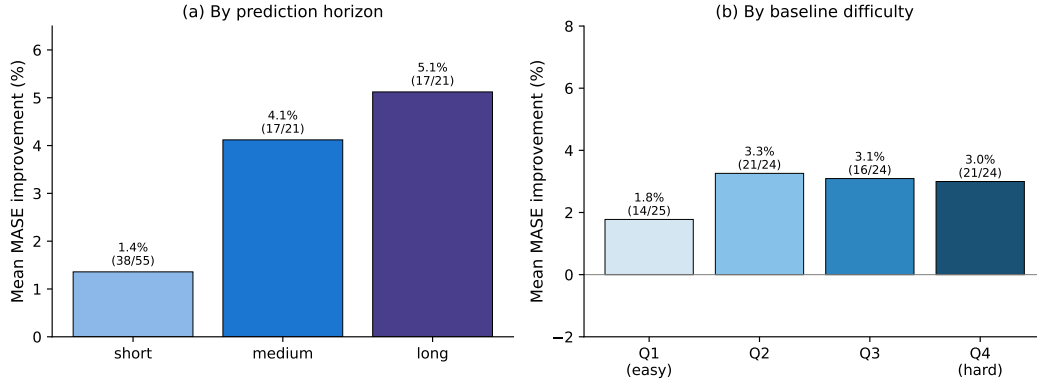


Figure 6: (a) Improvement by prediction horizon (short/medium/long). (b) Improvement by baseline difficulty quartile. Harder configurations benefit more.

Prediction length	HD10 $\Delta\%$	Win rate
30	-0.1%	57%
48	-2.1%	70%
480	-6.7%	68%
720	-8.6%	79%
900	-14.1%	100%

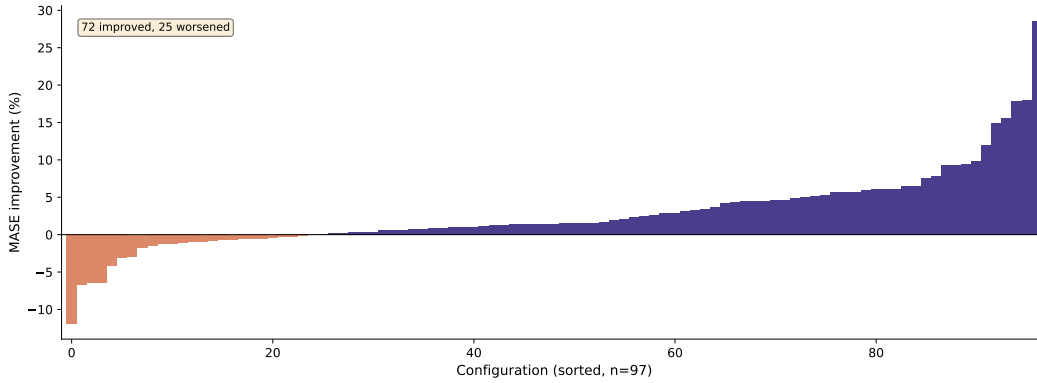


Figure 7: Per-configuration improvement (5-seed average), sorted. Green: HD10 wins. Red: baseline wins. 72/97 configurations improve.

Context	BL	HD10	$\Delta\%$	Wins/97
512	0.9432	0.8986	-4.73%	62
1,000	0.9166	0.8706	-5.02%	65
2,000	0.8813	0.8331	-5.47%	71
4,000	0.8666	0.8234	-4.99%	66

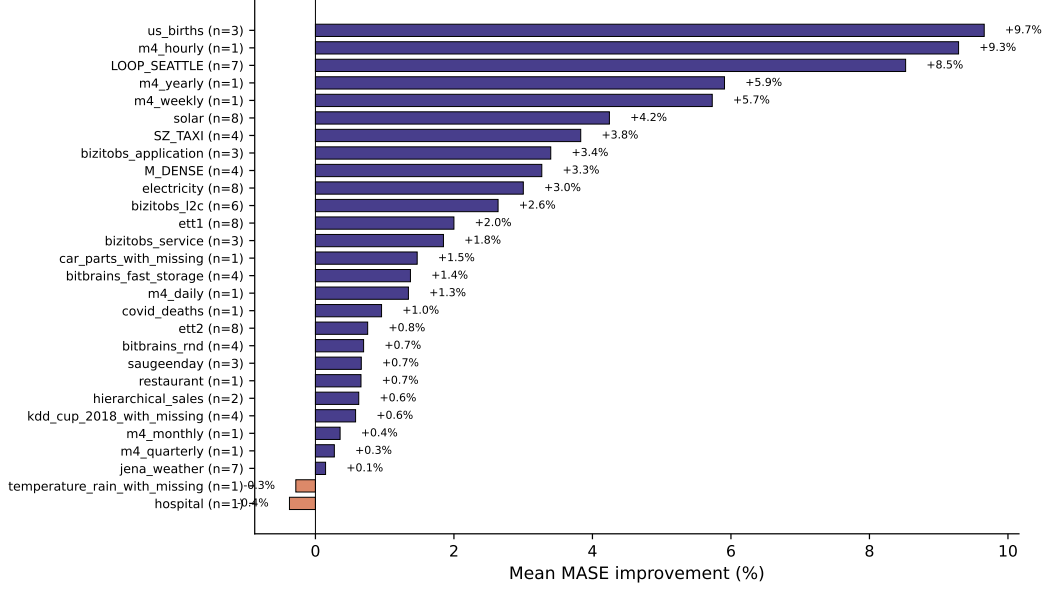


Figure 8: Improvement by dataset (5-seed average). Transport and energy datasets benefit most; weather is neutral.

Table 8: Final training loss across 5 seeds at 10K steps ($\eta = 10^{-3}$, epoch-averaged). Training loss differences between methods are small ($\sim 1\%$), while eval MASE differences are much larger ($\sim 3\%$), confirming hints act as a regularizer.

Method	s0	s1	s2	s7	s42	Mean	Std	Eval MASE
Baseline	.1366	.1365	.1358	.1366	.1365	.1364	.0003	0.8617
HD10 (ours)	.1354	.1350	.1350	.1353	.1343	.1350	.0004	0.8368
Zero	.1378	.1369	.1363	.1366	.1353	.1366	.0008	0.8578
Duplicate	.1376	.1366	.1370	.1365	.1355	.1366	.0007	0.8650
HD10 $\Delta\%$ vs BL	−1.0%							−2.9%

Table 9: Final training loss across seeds at 100K steps ($\eta = 10^{-3}$). HD10 maintains lower training loss on every seed. Zero and Duplicate have training loss comparable to baseline despite worse eval MASE.

Method	s0	s1	s2	s7	s42	Mean	Std	Eval MASE
Baseline	.1185	.1185	.1190	.1194	.1179	.1187	.0005	0.8784
HD10 (ours)	.1144	.1170	.1155	.1172	.1155	.1159	.0010	0.8443
Zero	.1186	.1188	.1184	.1188	.1187	.1187	.0002	0.8937
Duplicate	.1192	.1193	.1185	.1197	.1182	.1190	.0006	0.8860
MSHD10	.1147	—	.1145	.1145	—	.1146	.0001	0.8378 [†]
HD10 $\Delta\%$ vs BL	−2.4%							−3.9%

[†]MSHD10 eval mean over 3 available seeds (0, 2, 7).

Table 10: Train loss vs eval MASE improvement (5-seed mean, HD10 over baseline).

	Train loss $\Delta\%$	Eval MASE $\Delta\%$	Amplification
10K steps (5 seeds)	−1.0%	−2.9%	2.9×
100K steps (5 seeds)	−2.4%	−3.9%	1.6×

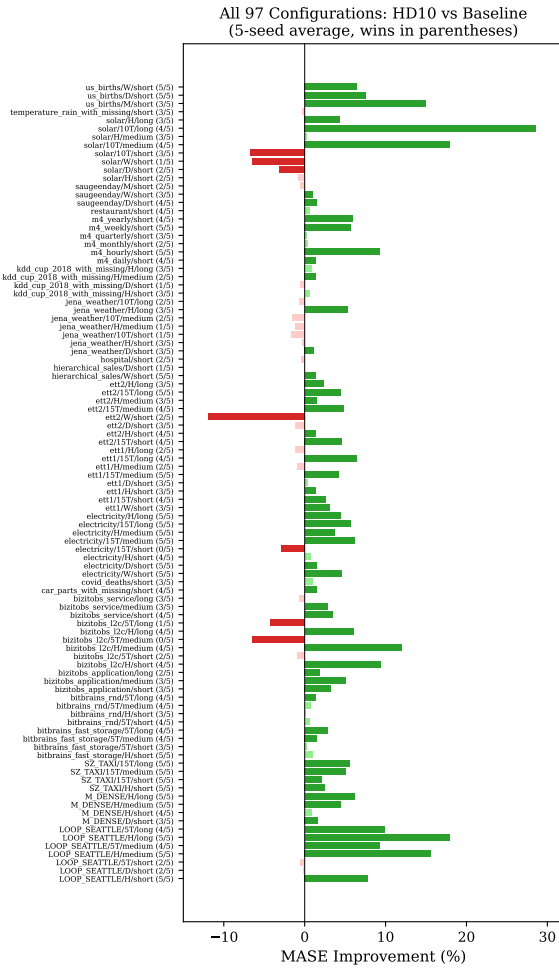


Figure 9: All 97 configurations: 5-seed averaged improvement and seed win count.

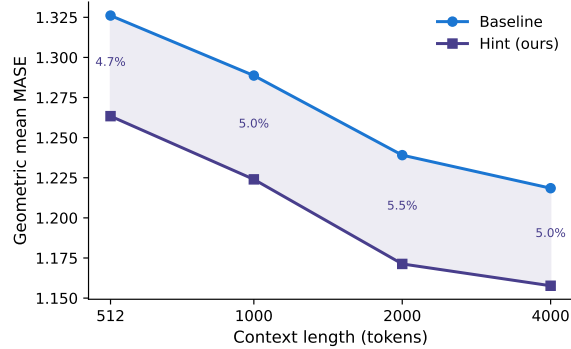


Figure 10: The improvement from hint preconditioning is approximately constant ($\sim 5\%$) across context lengths 512–4,000. HD10 at context 2,000 (0.8331) outperforms the baseline at 4,000 (0.8666).

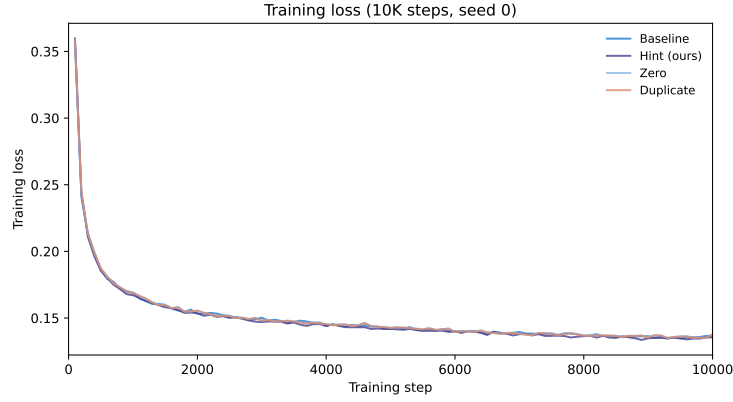


Figure 11: Training loss at 10K steps (seed 0). All methods converge similarly; the eval improvement from hints is a generalization effect.

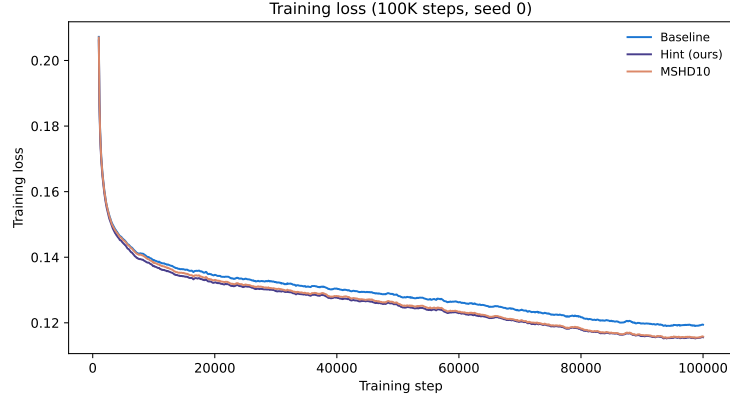


Figure 12: Training loss at 100K steps (seed 0). HD10 and MSHD10 maintain lower training loss throughout. The baseline’s eval collapse at 80K is not reflected in training loss.

Table 11: Training loss at matched LR $\eta = 2 \times 10^{-3}$ (5 seeds, 10K steps).

Method	s0	s1	s2	s7	s42	Mean
BL@2e-3	.1355	.1359	.1357	.1349	.1345	.1353
HD10@2e-3	.1346	.1341	.1340	.1337	.1331	.1339
$\Delta\%$	-0.7%	-1.3%	-1.2%	-0.9%	-1.1%	-1.0%

Table 12: HD10 vs baseline training loss gap over 100K steps (seed 0). The gap grows monotonically and does not plateau.

Epoch	100	5K	10K	30K	50K	70K	100K
BL loss	.3600	.1441	.1405	.1318	.1267	.1238	.1185
HD10 loss	.3641	.1428	.1387	.1293	.1240	.1205	.1144
Gap	-1.1%	+0.9%	+1.3%	+1.9%	+2.2%	+2.7%	+3.5%

Table 13: Base model (46M parameters, 10K steps). Results are inconsistent across seeds.

Seed	BL (46M)	HD10 (46M)	$\Delta\%$	Wins
0	0.8504	0.8520	−0.19%	52/95
1	0.8473	0.8509	−0.42%	60/95
42	0.8570	0.8417	+1.80%	59/93

Degree	Effective lags	MASE	$\Delta\%$ vs BL	Wins/97
2	1 (at $2P$)	0.8388	−3.20%	63
3	1 (at $3P$)	0.8497	−1.95%	55
4	2 (at $2P, 4P$)	0.8234	−4.99%	66
5	2 (at $P, 3P, 5P$)	0.8445	−2.56%	—
6	3 (at $2P, 4P, 6P$)	0.8345	−3.71%	69
7	3 (at $P, 3P, 5P, 7P$)	0.8397	−3.11%	—
8	4 (at $2P-8P$)	0.8413	−2.92%	62
0 (dup)	0	0.8583	−0.96%	42

Stride	Lookback	MASE	$\Delta\%$ vs BL	Wins/97
16 (= P)	32, 64 steps	0.8234	−4.99%	66
4	8, 16 steps	0.8406	−3.00%	57
8	16, 32 steps	0.8518	−1.71%	62
32	64, 128 steps	0.8484	−2.10%	63
8+16 (dual)	16, 32, 64 steps	0.8389	−3.20%	62

Basis	MASE	$\Delta\%$	Wins vs BL	p	vs Cheby	p
Chebyshev	0.8234	−4.99%	66/97	2.5×10^{-4}	—	—
EMA	0.8329	−3.89%	65/97	5.3×10^{-4}	41/97	0.077
Legendre $d=4$	0.8400	−3.07%	62/97	4.0×10^{-3}	30/97	<0.001
Legendre $d=6$	0.8397	−3.10%	64/97	1.1×10^{-3}	50/97	0.42
Finite Diff	0.8569	−1.12%	54/97	0.16	29/97	<0.001
Duplicate	0.8583	−0.96%	42/97	0.11	27/97	<0.001

Dropout	MASE	$\Delta\%$ vs BL	Wins/97	p
0% (HD0)	0.8378	−3.5%	—	—
2%	0.8388	−3.20%	63	0.004
3%	0.8497	−1.95%	55	0.22
10%	0.8234	−4.99%	66	0.0005
20%	0.8422	−2.81%	67	0.0002
30%	0.8559	−1.23%	61	0.014

Initialization	Learned coefficients	MASE	$\Delta\%$
Chebyshev init	$[-0.026, -1.032, -0.123, 0.007]$	0.8332	−3.85%
Zero init	$[0.308, 0.177, 0.057, 0.045]$	0.8358	−3.57%
Fixed Chebyshev	$[0, -2, 0, 0.5]$ (not learned)	0.8234	−4.99%

Method	Filter location	MASE	vs BL
HD10 (input-level)	Before patching	0.8234	−4.99%
Latent $s=1$	After input projection	1.1538	+33.1%
Latent $s=4$	After input projection	1.2243	+41.3%

Train dropout	Normal MASE	Zero-hint MASE	Degradation
0% (MS46)	0.8534	1.1594	+35.9%
2%	0.8388	0.8996	+7.2%
10% (HD10)	0.8234	0.9003	+9.3%
30%	0.8559	0.8769	+2.5%

	Baseline	HD10
Training throughput	2.18 it/s	2.14 it/s (−1.6%)
Extra parameters	0	12,288 (+0.11%)
Peak GPU memory	identical	identical

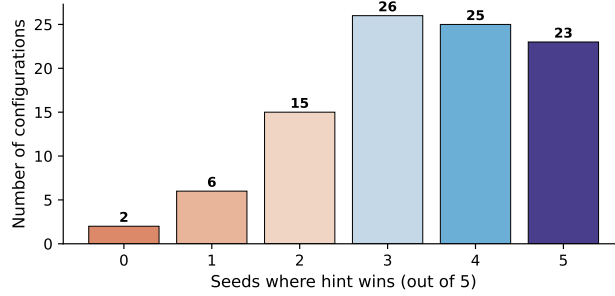


Figure 13: For each configuration, how many of 5 seeds does HD10 win? 74 of 97 configs see HD10 win the majority ($\geq 3/5$) of seeds.

Representative Configurations: Where HD10 Helps and Hurts

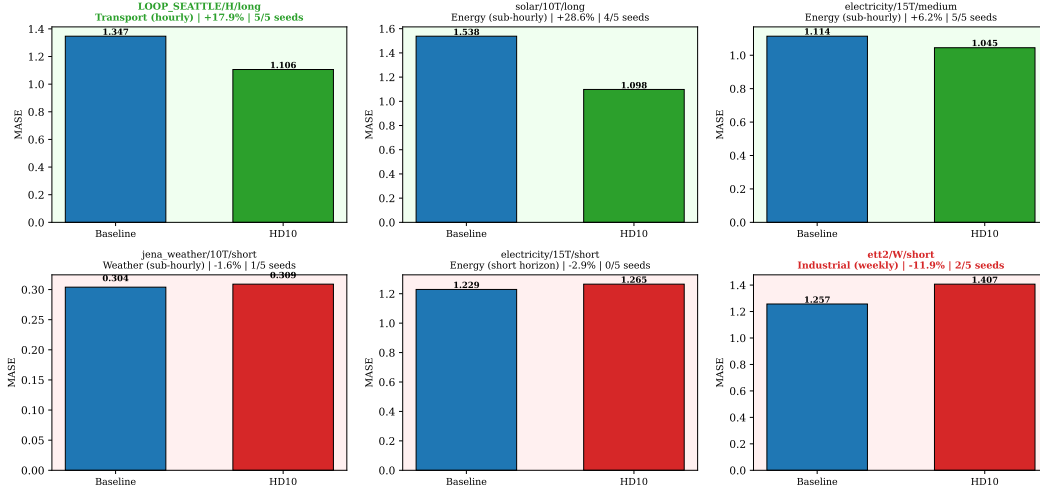


Figure 14: Six representative configurations. Top: strong wins (transport, energy). Bottom: neutral (weather) and losses.

Table 14: FEV-Bench results (100 tasks, seed 0, 10K steps). HD10 wins 72% of tasks on this independent benchmark.

Method	MASE (geo)	SQL (mean)	MASE wins	SQL wins
Baseline	1.2841	1.6786	—	—
HD10 (Ours)	1.2525	1.6572	72/100 (72%)	72/100 (72%)
$\Delta\%$	−2.5%	−1.3%		

Table 15: GIFT-Eval leaderboard context. Our HD10 at 11.4M params outperforms Moirai 1.0 Large at 311M.

Model	Params	Norm. MASE
Moirai 2.0-R-Small (official)	11.4M	0.728
Kairos-50M	53M	0.742
TimesFM-2.0	500M	0.758
ChronosBolt-Base	205M	0.808
Our HD10 (seed 0, 10K)	11.4M	0.828
PatchTST (leaderboard)	—	0.849
Our Baseline (seed 0, 10K)	11.4M	0.870
Moirai 1.0 Large	311M	0.875
Moirai 1.0 Base	90M	0.901
Moirai 1.0 Small	14M	0.946

Table 16: Matched-official 100K results (10K warmup, cosine annealing to 0, otherwise identical to official protocol).

Method	s0	s2	s7	Mean	vs BL
Baseline (1K warmup)	0.909	0.863	0.864	0.878	—
Baseline (10K warmup)	0.882	0.880	0.917	0.893	—
HD10 (1K warmup)	0.835	0.859	0.845	0.844	−3.9%
HD10 (10K warmup)	0.892	0.846	0.880	0.873	−2.3%